

Agentic AI in der Praxis

Vom Spec zum produktiven Workflow — mit Übungen am eigenen Rechner

IT4Science Days 2026 · Agentic AI Workshop · 09:00–12:00

Christian Uhl · Tobias Weiß

Wer wir sind: zwei Beispiele unserer Arbeit

Christian Uhl

Zentrum für angewandte Informatik & Data Science,
Uni Gießen

Aus Forschung und Lehre:

- *Ein unbekannter, kaum dokumentierter Chip ohne Firmware.*
- *Ein einzelner Fehler hätte ihn zerstört.*
- *Sorgfältig war zu langsam, schnell zu riskant.*
- *Nach 24 Stunden: eine **FIDO2-fähige Firmware, spezifikations- und testgetrieben.***

Tobias Weiß

DevOps Engineer,
Uni Marburg

Aus der Praxis im Rechenzentrum:

- *openDesk Edu: Digitale Dienste für Forschung, Lehre und Verwaltung.*
- *Wissens-Pipelines & IT-Betrieb agentisch.*
- *Specs, Contracts, Tests: Open-Source-Software mit Open-Source-Werkzeugen erstellen.*

Ihre Reihe: Blitzlicht

- **Name · Fachrichtung · ein Thema**, zu dem ein Agent für Sie eine Literaturrecherche durchführen soll.
- **Selbsteinschätzung KI**: eher Anfänger oder eher Fortgeschritten?
- ≤ 30 Sekunden je Person, wir sammeln Wünsche an die Whiteboard-Wand.
- Diese Wünsche checken wir am Ende gegen die Outcomes.

Was der Vormittag liefert — und was nicht

- ein Research-Repo mit laufender Pipeline
- einen Spec-Change, live umgesetzt
- Werkzeuge für Modelle und Harnesses statt Trendlisten

Was nicht geliefert wird: der fertige Bot.

Wir bauen keine voll-autonomen Assistenten, sondern führen und korrigieren die Agenten.

Agenda (3h)

Block	Wer
Ankommen, Vorstellung (Praxisbeispiele + TN-Runde), Ablauf	Christian + Tobias
Grundlagen & Definition	Christian
Foundation Models: aktuelle Entwicklungen	Christian + Tobias
Open-Source Toolbox	Tobias + Christian
 Pause 10:10–10:20	
Anwendung 1: Ihr Thema direkt agentisch — ohne Spec	Tobias
Theorie: Spezifikation & Token	Christian + Tobias
Anwendung 2: Ihr Thema via Spec aufs Demo-Repo	Christian
 Pause 11:25–11:30	
Outcomes: TN präsentieren ihre Ergebnisse	Tobias
Q&A, Diskussion, Wrap Up	Christian + Tobias

Begriffe, bei denen wir uns festlegen

Begriff	Worüber gestritten wird	Unsere Festlegung für heute
Agent	von „Chatbot mit Schleife“ bis „autonomes System“	Das Modell bestimmt die nächsten Schritte. Feste Abfolge = Workflow.
Reasoning	ob das „Denken“ ist, ist offen	mehr Rechenzeit zur Laufzeit; keine Aussage über Kognition
Halluzination	Begriff selbst umstritten (eher: Konfabulation)	unbelegte Ausgabe — der Grund für die Testebene
Open Source	offene Gewichte sind nicht Open Source (OSI)	wir meinen offene Gewichte
Reproduzierbarkeit	gleiche Eingabe, abweichendes Ergebnis	geprüft wird das Ergebnis , nicht der Weg dorthin

Agentische Systeme und deren Bausteine

Baustein	Was es ist	Was es nicht ist
Modell (LLM)	Gewichte; sagt das jeweils nächste Token voraus	kein Wissensspeicher, keine Datenbank
Kontext	was im Eingabefenster steht: Repo, Spezifikation, Verlauf	kein Gedächtnis; endet mit der Sitzung
Werkzeuge	Zugriff auf Dateien, Shell, Netz, Schnittstellen	keine Rechte jenseits des Freigegebenen
Harness	führt aus, was das Modell verlangt: Kontext stellen, Werkzeuge aufrufen, Ergebnisse zurückspeisen, Schleife beenden	trifft die Entscheidungen nicht selbst

Spezifikation · Vertrag · Tests

**Die Struktur, auf die wir den ganzen Vormittag
zurückkommen**

Fragen oder Beauftragen

- **Fragen:** „Fassen Sie diese Papers zusammen.“
- **Beauftragen:** „Extrahieren Sie diese fünf Felder aus jedem Paper.“

 Fragen gegen Beauftragen: nur das Beauftragen hat eine Prüfung mit Abbruchbedingung

Eine Metapher für den ganzen Workshop

 Spec · Contract · Test Pyramide

Was bedeutet die Pyramide?

Ebene	Legt fest	Im Repository
Spezifikation	Was gelten soll, als überprüfbare Anforderung formuliert	<code>specs/</code> , <code>AGENTS.md</code> : die verbindlichen Regeln
Vertrag	Was sich in genau diesem Schritt ändert, und woran man das misst	ein Change: Vorschlag, Entwurf, Spezifikation, Aufgabenliste
Tests	Ob die Anforderung erfüllt ist, ohne Ermessensspielraum	Testsuite, Validierung, automatische Prüfung bei jeder Änderung

Erledigt ist eine Aufgabe erst, wenn die Prüfung bestanden ist.

Foundation Models: aktuelle Entwicklungen

Wo stehen die Modelle 2026? Sechs Entwicklungen

1. Reasoning reif: planen statt komplettieren
2. Kontext explodiert: 200K → 1M+ Token
3. MCP wird Standard: „USB-C der Tools“
4. Tool-Use produktionsreif: strukturiert, parallel
5. Offene Gewichte holen auf: Kosten-Kollaps
6. Lokal & souverän: DSGVO ohne Qualitätsverlust

SAIA-Katalog (09/2026): GLM 4.7 · GLM 5.3 Flash · Qwen 3.5 (122–397B MoE) · Qwen 3.6 35B · Qwen3.8 27B · Qwen3-Coder-Next · Devstral 2 · DeepSeek V4 Flash · GPT-OSS 120B · Gemma 4 31B

Foundation Models: Praxis-Einordnung

Aufgabe	Derzeit stark	Anmerkung
Freie Textproduktion, Übersetzung	GPT-6	derzeit führend bei Prosa
Räumliche Aufgaben, 3D-Modellierung	GPT-6	in eigenen Tests überzeugend
Code schreiben, größere Umbauten	Claude , zunehmend GLM	Claude teuer, aber oft auf Anhieb erfolgreich
Code gezielt umschreiben	Qwen3.8-Flash-Next , GLM	offene Gewichte, schnell, günstig
Abläufe mit Werkzeugen steuern	GLM 4.7 , Claude	verlässliche Werkzeugaufrufe
Sensible Forschungsdaten	lokal : Ollama, vLLM, llama.cpp	die Daten verlassen das Haus nicht
Viele gleichartige Fälle	kleine Modelle, z. B. DeepSeek V4 Flash	Geschwindigkeit und Kosten entscheiden

Die offenen Modelle holen massiv auf. In vielen Aufgaben ist der Abstand inzwischen kleiner als der Preisunterschied.

Quelle: eigene Erfassung

Das Harness bestimmt, was das Modell erreichen kann

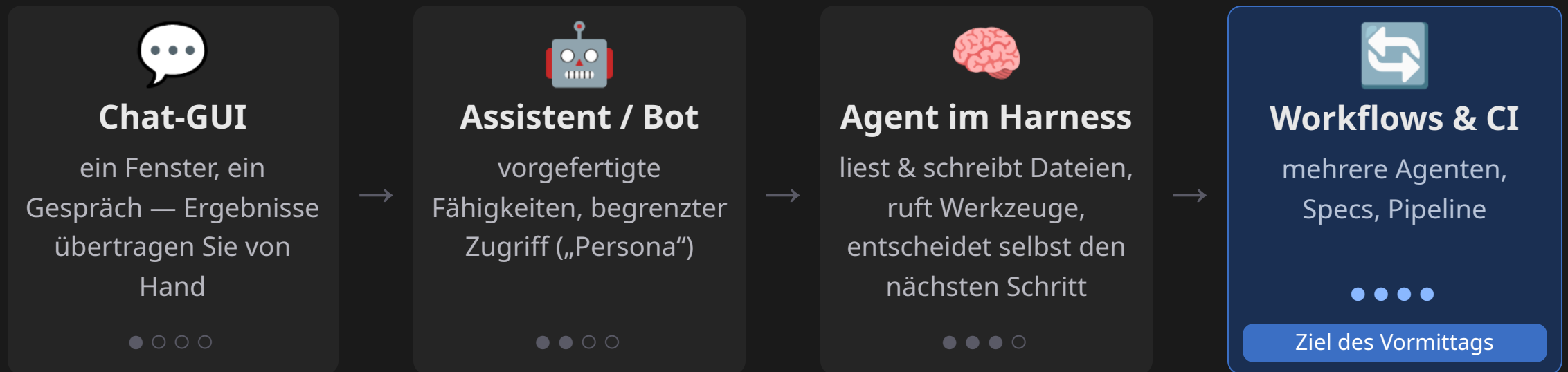
- Einzelne Frage → das Modell prägt die Antwort allein
- Mehrere Schritte, Werkzeuge, Selbstkorrektur → das Harness setzt die Reichweite
- Jeden Schritt darin wählt weiterhin das Modell
- Günstiges Modell im guten Harness vor teurem im schlechten (eigene Beobachtung)

Auch das beste Modell ist nur so gut wie sein Harness.

Open-Source Toolbox

Harnesses: die Steuerungsebene über dem Modell

Von der Chat-Webseite zum Terminal-Agenten



Autonomie wächst nach rechts: selbst entscheiden, selbst handeln

Agentisch ist nicht das Terminal — es ist die Autonomie.

Auch eine GUI kann agentisch sein; das Terminal ist heute der reifste Ort dafür: prüfbar, wiederholbar, automatisierbar.

Zwei Harnesses im Vergleich

1. OpenCode: der Allrounder

- Agent im Terminal (CLI/TUI), Modell je Agent.
- LSP, Plugins, Skills, MCP — omO: Routing je Aufgabe, AST-Grep, Background-Agents.

2. pi: der Minimalist

- Skills · Prompt-Templates · Packages · Themes.
- „Adapt pi, nicht umgekehrt“.

Einsatz entscheidet:

Einsatz	Harness
tägliche Coding-Agents, Modell-Routing	OpenCode
kontrollierte, minimalistische Workflows	pi

*Ein **Harness** ist die **Steuerungsebene**: gleiche Modelle, unterschiedliche Ergebnisse, je nach Harness.*

Kommerzielle Gegenprobe — Claude Code · Codex · Antigravity CLI:
bequem, sofort brauchbar — aber anbieter-optimiert, im Mittelfeld.

Quelle: eigene Erfassung

pi als Plattform: Packages statt Features

Kategorie	Beispiele (Auswahl)
Skills	superpowers (TDD, Debugging) · CVE-Scanner · AGENTS.md-Audit
Agenten & Orchestrierung	subagents (Council-Modus) · until-done (Ziel-Schleife) · plan-mode · evaluate (Gegenspieler-Prüfung)
Output-Stil, Werkzeuge & Suche	caveman (knapp) · ponytail (minimal) · mcp-Adapter (LibreOffice, n8n) · websearch · searxng
UI	webui · desktop-ui · glimpseui (native Dialoge) · chrome
Absicherung	tool-repair · retry · permission-system · secrets · quotas · rtk

Der Kern bleibt klein — alles andere ist nachrüstbar.

26 Packages · pi.dev/packages · eigene Installation, Stand 09/2026



10 Minuten.

Hands-On: Erste Schritte

Ihr Thema X, direkt agentisch, ohne Spec

Erst planen, dann bauen: die Modes

- **Plan-Mode:** der Agent liest nur — stellt Fragen, schlägt Schritte vor. **Nichts** wird geändert.
- **Build-Mode:** der Agent setzt den freigegebenen Plan um
- Plan lesen ist der schnellste Weg, agentisches Arbeiten zu verstehen — genau so starten wir gleich

Übung 1: Direkt agentisch — ohne Spec

~15 Minuten im Agenten (opencode / pi mit SAIA): stellen Sie Ihr Thema X aus dem Blitzlicht. Was kommt dabei heraus?

Der Agent übernimmt	Heißt konkret
Entdecken	neue Veröffentlichungen zu Thema X aufspüren
Einordnen	Funde den Themenfeldern zuordnen
Prüfen	Quellen verifizieren, Erfindungen aussortieren
Berichten	Überblick, Trends, Kurz-Briefings pflegen

Sie entscheiden, was relevant ist. Der Agent übernimmt die Fleißarbeit. Heben Sie Ihre Treffer auf, in Übung 2 arbeiten Sie damit weiter.

AGENTS.md: der Contract für den Agenten

Die Regeln, die der Agent bei jedem Start liest:

- **Erzeugtes wird nie von Hand geändert** — die Pipeline regeneriert es.
- **Nichts wird erfunden** — jede Quelle muss real auflösbar sein.
- **Nach jeder Änderung: komplette Prüfung**, erst dann Commit.

Die Spec ist der Vertrag. Der Agent wird nicht im Prompt gefragt, sondern per Datei geführt: stabil, wiederholbar, review-bar. Zuwiderhandlungen deckt die Pipeline objektiv auf.

Spezifikation und Token

Präzise Verträge schreiben, sparsam mit Kontext umgehen

Wo die drei Ebenen im Projekt liegen

Ort	Was dort liegt	Rolle
<code>openspec/specs/</code>	das dauerhaft gültige Verhalten	Spezifikation: gilt, bis sie geändert wird
<code>openspec/changes/<name>/</code>	die eine Änderung, die gerade ansteht	Vertrag: genau das bekommt der Agent
<code>openspec/archive/</code>	alle abgeschlossenen Änderungen	Begründung: warum wurde das so entschieden

*Der Agent braucht nicht das ganze Projekt, der Vertrag genügt.
Das ist zugleich die wirksamste Einsparung an Kontext.*

Zwei Schulen: Spec-Kit und OpenSpec

	Spec-Kit	OpenSpec
Grundidee	beschreiben, was gebaut werden soll	beschreiben, wie sich Bestehendes ändert
Typischer Fall	Neubau, neue Funktionen	laufende Weiterentwicklung
Quelle der Wahrheit	eine Spezifikation je Vorhaben	eine dauerhafte Spezifikation plus Änderungen
Ablauf	specify · clarify · plan · tasks · implement	propose · apply · sync · archive
Aufwand	höher, dafür mehr Prozessführung	niedriger

Neues Vorhaben: Spec-Kit. Bestehendes System, das wächst: OpenSpec.

Wir bearbeiten heute ein bestehendes Repository, deshalb OpenSpec.

OpenSpec-Showcase: ai-literacy-research

Ein live implementierter Change steht bereit, im Repo `ai-literacy-research`.

- **Corpus:** 9.600+ Papers, 20 Kategorien, lebende systematische Übersicht.
- **OpenSpec-Struktur** am echten Beispiel:
 - `openspec/specs/`: paper-corpus · discovery-pipeline · analysis-reporting (Source of Truth)
 - `openspec/changes/archive/2026-08-23-add-research-gap-analysis/`:
proposal → design → specs → tasks

Gap-Score = 0.6 × Dünne (thinness) + 0.4 × Momentum + Floor-Bonus
→ rankt Bereiche mit wenig Literatur und starkem Wachstum.

Weniger Token, gleiche Qualität

Praktisch gemessen: 3–5× weniger Token bei gleicher Ergebnisqualität, zwei Hebel.

Routing: das richtige Modell pro Aufgabe

- omO: benannte Agents → Modell je Kategorie.
- LiteLLM-Gateway: ein Endpunkt, viele Modelle — Wahl nach Aufgabe.

Caching & Kontext-Hygiene

- Prompt-/KV-Caching, Kompaktion: nur behalten, was zählt.

Klein ist schön. Ein kleines Modell für eine kleine Aufgabe. Nicht jede Aufgabe braucht den größten Verstand.

Komponiere, nicht wiederhole. Skills & Specs vorschreiben, statt in jeder Session alles neu zu erklären.

Speichere Wissen, nicht Prompt-Stücke. pi-memory / Knowledge Graph persistieren Erfahrung außerhalb des Kontexts.

Hands-On: Mit Spec

Ihr Thema X, via Spec aufs Demo-Repo

Ihr Thema X als Spec — skeleton-research

Jeder verlässt den Raum mit einem eigenen, CI-validierten Research-Repo.

Gleiche Frage wie in Übung 1 — aber Spec statt Sammel-Haufen.

- **Corpus statt Haufen:** das Wissen zum Thema als geprüfter Text-Bestand.
- **Regeln statt Handarbeit:** prüfen, erzeugen, berichten — als eine Pipeline.
- **Lebendig statt Momentaufnahme:** der Dienst erweitert und prüft wöchentlich selbst.

Die Pipeline: jede Stufe hat einen Zweck

Sammeln → Prüfen → Erzeugen → Veröffentlichen

Ein Tool, ein Job. Jedes Skript tut genau eine Sache (validate / generate / stats / reports), und sie komponieren sich zur Pipeline.

Text ist die Schnittstelle. YAML + Markdown, kein GUI nötig. Ein Mensch kann genauso lesen wie ein Agent.

Die Pipeline ist der Richter. Vom Agenten erzeugte Edits müssen die Validierung bestehen, objektiv und nicht nach Meinung.

Jump-Start in 5 Schritten

1. **Kopie holen:** das Demo-Repo wird Ihres.
2. **Themenfeld setzen:** Ihre Ordnung für Thema X.
3. **Seeden:** 3–5 Treffer aus Ihrer Übung-1-Runde — die Prüfung entscheidet, was überlebt.
4. **Laufen lassen:** prüfen, erzeugen, berichten.
5. **Veröffentlichen:** das Ergebnis geht online — und bleibt am Leben.

Was die Pipeline erzeugt, wird nie von Hand geändert — es regeneriert sich aus dem Corpus.

Übung 2: Ihre eigene Auswertung

- **Ziel:** eine überprüfbare Anforderung für Ihr Thema X — auf dem Repo aus Teil 1.
- **Klein halten:** eine Auswertung, eine Kennzahl. Nicht das ganze Vorhaben.
- **Fertig heißt:** die Prüfung läuft durch. Nicht: der Agent meldet Vollzug.

```
npm install -g openspec          # falls noch nicht vorhanden
openspec new change thema-x      # Ihr Thema aus Teil 1
openspec validate --changes      # entscheidet, ob es zählt
```

Ohne OpenSpec genauso möglich: dieselbe Struktur als Markdown-Datei.

Übung 2: Woran Sie Ihr Ergebnis messen

Ebene	Mindestziel	Vollständig
Spezifikation	ein Satz, der widerlegt werden kann	Szenario mit Ausgangslage, Eingriff, erwartetem Ergebnis
Vertrag	eine Aufgabenliste mit drei Punkten	Vorschlag, Entwurf, Spezifikation, Aufgaben
Tests	eine Prüfung, die fehlschlagen könnte	die Prüfung läuft automatisch und besteht

Erledigt ist, was die Prüfung besteht.

Wer nur das Mindestziel erreicht, hat die Methode trotzdem verstanden.



5 Minuten.

Outcomes: Ihre Ergebnisse

Outcomes: Das zeigen wir

- 3–4 Freiwillige, je 3–4 min: eigenes Research-Repo + eigener Mini-Change.
- Live-Demo oder Screenshot, Hauptsache **Pipeline grün, Change validiert**.
- Bewertung an der Pyramide: ein SHALL-Satz (Spec), ein Change (Contract), ein grüner Check (Test).

Das Publikum reviewed mit: Was würde die Pyramide an Ihrem Projekt bemängeln?

Q&A · Diskussion · Wrap Up

Das Muster auf eigene Projekte übertragen

1. **Klein starten:** eine Spec für ein vorhandenes Modul (Given/When/Then).
2. **Delta-Specs nutzen:** erster Change mit 3 Tasks → Agent implementieren lassen.
3. **Verifikation erzwingen:** Tests + CI als Pflicht: Agenten iterieren, bis grün.
4. **Wissen persistieren:** pi-memory / Knowledge Graph ab Session 1, nicht in Prompt-Stücken.
5. **Loop institutionalisieren:** Betriebserkenntnisse werden neue Changes (Spec evolves).

Die Pyramide ist überall: papers.yaml = Spec · Pipeline = Contract · CI = Test.

Resources

- **OpenCode:** github.com/sst/opencode
- **pi:** pi.dev · [@earendil-works/pi-coding-agent](https://github.com/earendil-works/pi-coding-agent)
- **zot:** zot.sh · github.com/patriceckhart/zot
- **OpenSpec:** npmjs.com/package/openspec
- **SAIA-Plugins:** codeberg.org/tobias-weiss-ai-xr/pi-saia-plugin · github.com/tobias-weiss-ai-xr/opencode-saia-plugin · [zot-saia-plugin](https://github.com/tobias-weiss-ai-xr/zot-saia-plugin)
- **oh-my-opencode:** github.com/code-yeongyu/oh-my-opencode
- **skeleton-research:** github.com/tobias-weiss-ai-xr/skeleton-research
- **ai-literacy-research:** github.com/tobias-weiss-ai-xr/ai-literacy-research
- **Superpowers:** github.com/obra/superpowers
- **rtk:** github.com/rtk-ai/rtk
- **ponytail:** github.com/DietrichGebert/ponytail · **caveman:** github.com/JuliusBrussee/caveman

Vielen Dank!

Materialien & Folien: dieses Repo. Fragen danach gern als GitHub-Issue.

License

This work is licensed under a [Creative Commons Attribution 4.0 International License](#).

Christian Uhl · Tobias Weiß, except where otherwise noted.